

4 TECHNICAL OVERVIEW

This section of the document describes the full ALMA simulation pipeline implemented across the modules `run_simulation_pipeline.py`, `qol_pipeline.py`, `pb_vp.py`, `generate_builtin_sky_model.py`, `utils.py`, and `validation.py`. The pipeline supports both homogeneous and heterogeneous ALMA arrays, full per-antenna voltage-pattern (VP) modelling, atmospheric noise simulation, CLEAN/dirty imaging, baseline-group imaging, and noise-validation. The goal is to provide a clear, concise technical summary of how the system works and how it should be used.

Pipeline Architecture

The pipeline contains five major subsystems, each implemented primarily in one or more Python modules:

1. **Main** (`run_simulation_pipeline.py`): top-level driver that reads the user / example settings, resolves ALMA configuration files, computes imaging geometry, generates or imports the sky model, decides between heterogeneous vs homogeneous simulation paths, calls the appropriate MS-creation + imaging + validation routines, and finally writes a structured `pipeline_log.json` summary for the run.
2. **Simulation & Imaging Core** (`qol_pipeline.py`): low-level CASA utilities that actually build custom MSs, construct hour-angle scan patterns, apply the `tsys-atm` noise model with per-diameter efficiency scaling, predict the sky model into `MODEL_DATA`, add it to the noisy `DATA`, estimate SEFDs from the visibilities, compute array-dependent imaging geometry (beam, cell, imsize, hetero vs homo), and provide robust image-statistics tools (PB-aware RMS, peak, restoring beam, baseline grouping, etc.).
3. **Built-in Sky Model Generator** (`generate_builtin_sky_model.py`): standalone CASA script used by the pipeline when `sky_mode="builtin"`. It generates two synthetic test sources on the requested grid: (i) a “ring” model with a bright central Gaussian and eight ring Gaussians that are each 100x fainter than the centre, and (ii) a “point+disk” model with a bright central point source plus a fainter compact disk. The script can be driven either from JSON configuration or from command-line arguments.
4. **PB/VP System** (`pb_vp.py`): tools for primary-beam and voltage-pattern handling. This module computes λ/D -based PB sizes, generates per-dish PB images (ALMA Gaussian, Airy, or custom Gaussian) on a common grid with guard rings, groups antennas by diameter, and assembles a per-antenna VP table that `tclean` can use for AW-projection in both heterogeneous and “PB-aware” homogeneous imaging.
5. **Imaging/Stats Helpers & General Utils** (`utils.py` and `qol_pipeline.py` and): convenience layer around CASA imaging. This includes FFT-friendly imsize selection, dirty and CLEAN imaging wrappers, automatic multiscale setup from the PSF, central-box mask generation, robust PB-masked RMS estimators, baseline-group imaging (A/B/cross), as well as generic helpers for logging, safe removal of CASA products, and short, descriptive imagename tags.
6. **Noise Validation & Heterogeneity Checks** (`validation.py`): validation suite that copies the science MS into a noise-only sandbox, subtracts the sky model (or plain `simobserve` MS) from the `DATA` column, constructs A/B/cross sub-MSs where relevant, computes theoretical σ_{vis} and σ_{im} for each baseline group, runs natural-weight mosaic dirty imaging, measures PB-masked image RMS with the same machinery as the main pipeline, and prints/tabulates measured vs theoretical noise plus per-baseline-type diagnostics for heterogeneous arrays.

End-to-End Workflow

The complete pipeline executes the following sequence:

1. Load user settings and global configuration.
2. Locate ALMA configuration file and parse antenna positions & dish diameters.
3. Automatically compute imaging geometry: beam FWHM, cell size, FFT-efficient imsize.

4. Generate sky model via CASA script and import ⁴.
5. Create Measurement Set (MS): heterogeneous (custom) or homogeneous (simobserve).
6. Generate per-diameter PB images and construct per-antenna VP table.
7. Apply atmospheric noise (tsys-atm) and per-diameter efficiency scaling.
8. Predict sky model (MODEL_DATA) and add to noisy visibilities.
9. Perform dirty imaging, CLEAN if requested.
10. Optionally image baseline groups (A/A, B/B, A/B).
11. Run noise-validation imaging for heterogeneous configurations.
12. Export FITS & log results.

4.1 *Heterogeneous vs Homogeneous Arrays*

Dish diameters parsed from ALMA `.cfg` files determine the simulation mode.

4.1.1 Homogeneous Arrays

- One single dish size.
- MS generated with `simobserve`.
- Optional VP table for PB-aware imaging (only ever needed when simulating homogeneous arrays with other dish sizes than 7 m and 12 m!).
- Uniform efficiency noise scaling.

4.1.2 Heterogeneous Arrays

- Multiple dish sizes present (e.g., 12m+25m).
- Custom MS preserving per-antenna diameters.
- PB/VP image per diameter.
- Per-baseline noise scaling for correct SEFD ratios.
- Baseline-group imaging & validation.

4.2 *Primary-Beam & Voltage-Pattern System*

The PB/VP system generates accurate per-antenna PB models:

1. Compute PB FWHM via ALMA or analytic models.
2. Generate PB images (voltage form, XX/YY) with guard ring⁵.
3. Construct VP table with `vpmanager.setpbimage`.
4. Enable AW-projection in `tclean` for heterogeneous imaging.

⁴It is possible to supply your own skymodel/source as a fits file, as when normally simulating with CASA. A "skyscript" is just a script which crates a skymodel/source and base the dimensions of it upon the input parameters in the pipeline, it can therefore dynamically adjust the dimensions based on the configuration used.

⁵**Why XX/YY?** CASA's voltage-pattern tables expect per-polarization primary beams. For ALMA, only the parallel-hand feeds (XX, YY) are used for PB correction; cross-hands (XY, YX) are set to zero. **Guard ring:** PB values near the image boundary are forced to zero to avoid edge artifacts and numerical instabilities in AW-projection.

4.3 Built-in Sky Models

The pipeline includes two synthetic sky models implemented in `generate_builtin_sky_model.py`. These models provide controlled, reproducible test cases without requiring external FITS files.

- **9-Gaussian Ring Model** This model consists of nine concentric Gaussian components: a central compact Gaussian that is 100 times brighter than the surrounding rings, plus eight progressively larger, fainter Gaussians arranged radially. The result is a bright compact core embedded in smooth extended emission, ideal for testing dynamic-range limits, multiscale CLEAN behaviour, and PB-correction performance.
- **Point+Disk Model** A single unresolved point source at the image center combined with a circular Gaussian disk of adjustable size and flux ratio. This model is useful for validating flux recovery, beam characterization, and deconvolution behaviour on mixed compact–extended emission.

Both models are generated as single-channel CASA images using the pipeline’s imaging geometry (cell size, imsize, reference frequency), ensuring that their angular scales and flux distributions remain consistent with the user’s configuration.

4.4 Noise Injection

Noise injection is a two-step process:

1. Atmospheric noise (tsys-atm) using reference efficiency.
2. Per-diameter scaling: for baseline $i-j$,

$$f_{ij} = \frac{\eta_{\text{ref}}}{\sqrt{\eta_i \eta_j}}.$$

Extra Gaussian noise added if $f > 1$, DATA scaled if $f < 1$.

4.5 Imaging

1. Dirty imaging using mosaic gridding & AW-projection⁶.
2. Compute RMS & CLEAN threshold.
3. Optional CLEAN with multiscale (with dynamically scales based upon the beam size)⁷ and custom mask.
4. Measure peak, RMS, beam size.

4.5.1 Baseline-Group Imaging

For heterogeneous arrays, A, B, and cross baselines are imaged separately (if turned on). Dirty and optional CLEAN images are produced and validated.

4.6 Noise Validation

1. Copy MS and subtract model → noise-only MS.
2. Build A, B, cross sub-MSs.
3. Compute theoretical σ_{im} per group.
4. Combine groups via inverse-variance weights.
5. Perform natural-weight dirty imaging and measure RMS.
6. Compare with theoretical predictions.

⁶Mosaic gridding and AW-projection is needed for accurate heterogeneous array simulations independently from the number of pointings used! It enables PB aware imaging, which will use the custom PBs and VPs one created.

⁷Multiscale with dynamically set scales is recommended; although, the pipeline allows for any deconvolver to be used!

4.7 Running the Pipeline

Edit `EXAMPLE_SETTINGS` and run:

```
casa --nogui --nologger -c run_simulation_pipeline.py
```

All MSs, FITS, VP tables, logs, and JSON summaries are saved in `project_base`.

4.7.1 Example Configuration: `EXAMPLE_SETTINGS`

The publicly available main script is currently controlled through a single Python dictionary, `EXAMPLE_SETTINGS` (found at the top of `run_simulation_pipeline.py`), which groups all user-facing parameters in one place:

```
EXAMPLE_SETTINGS = dict(

    # --- Paths / project structure ---
    project_base="ALMA.validate",          # Folder where all outputs go
    casa_bin="/home/hampus/casa-6.7.0-31-py3.10.el8/bin/casa", # CASA executable
    extra_dirs=[],                          # Extra dirs to search for cfg files

    # --- Array configuration(s) ---
    config_map={"alma.H200.cfg": "GG"},    # {cfg_file : short_tag}
    declinations=["-23d00m00.00"],         # Target declination(s)
    integration_times=["1h"],              # On-source time(s)

    # --- Frequency setup ---
    center_freq="343.5GHz",                # Reference frequency
    width="7.5GHz",                        # Total bandwidth
    pwv=1.262,                             # PWV for ATM noise

    # --- Imaging / sky setup ---
    phasecenter_ra="12h00m00.00s",         # Phase center RA
    fov_arcsec=1.28,                       # Desired field of view (arcsec)
    sampling_factor=5.0,                   # Beam/cell sampling (higher -> smaller pixels)

    # --- Sky model selection ---
    # sky_mode controls *where* the sky model comes from:
    # "builtin" : use one of the internal test skies (sky_type below)
    # "fits"    : use a user-supplied FITS image directly
    sky_mode="builtin",

    # If sky_mode == "builtin", choose between:
    # "ring"    : 8+1 Gaussian ring (central Gaussian + 8 around, ring Gaussians are 100x fainter)
    # "pointdisk" : bright central point source + faint small disk
    sky_type="ring",

    # If sky_mode == "fits", this must point to an existing FITS file
    # that will be imported and used as the sky brightness model.
    sky_fits=None,                         # e.g. "/path/to/my_sky_model.fits"

    # --- Built-in sky parameters (only used for sky_mode="builtin") ---
    # Ring model (sky_type="ring"):
    ring_flux=30e-3,                       # Jy of central Gaussian (set VERY low for an
    # essentially noise-only image!)
    ring_n_sources=8,                      # Number of Gaussians in the ring
    ring_radius_as=0.15,                   # Ring radius in arcsec
    ring_major_as=0.035,                   # Major axis of each Gaussian (arcsec)
    ring_minor_as=0.035,                   # Minor axis of each Gaussian (arcsec)
    ring_pa_deg=0.0,                       # Position angle of Gaussians (deg)

    # Point+disk model (sky_type="pointdisk"):
    pnd_flux=1.0,                          # Base total flux (Jy) (set VERY low for an
    # essentially noise-only image!)
    pnd_point_flux=None,                   # If None -> uses pnd_flux
    pnd_disk_flux=None,                    # If None -> uses 0.1 * pnd_flux
    pnd_disk_diam_as=0.10,                 # Disk diameter (arcsec),

    # --- Optional manual geometry overrides ---
    # The pipeline normally computes "good" imaging geometry automatically
    # (cell size + image size) from the array beam and fov_arcsec.
    # These two knobs let you override that behaviour:
    #
    # * If cell_arcsec is not None:
```

```

# - this fixed cell size (arcsec / pixel) is used for BOTH
# the sky model and all imaging.
# * If imsize_manual is not None:
# - this fixed image size (pixels) is used for BOTH the
# sky model and all imaging.
#
# If either is None, the auto-geometry from compute_imaging_geometry()
# is used for that parameter instead.
cell_arcsec=None, # Fixed cell size (arcsec), overrides auto if set
imsize_manual=None, # Fixed image size (pixels), overrides auto if set

# --- tclean / deconvolution ---
weightings=["briggs"], # Weighting scheme(s)
robust_values=[0.5], # Briggs robust parameter(s)
uvtaper_values=["0.0arcsec"], # UV taper(s)
niter=500, # CLEAN iterations (0 = dirty)
deconvolver="multiscale", # Deconvolver type
threshold_factor=2, # CLEAN threshold = factor x dirty RMS

# global pblimit used for all tclean calls that specify pblimit
pblimit=0.2,
# Cleaning mask settings
clean_use_mask=True, # If True, use a central box mask for CLEAN (only
if niter>1)
clean_mask_center_frac=0.1, # Fraction of image size to keep as central mask

# --- Noise / Primary Beam ---
noise_mode="atm", # 'atm', 'manual', or 'none' (atm is HIGHLY
recommended)

# --- VP / PB usage for homogeneous configs ---
use_vptable_in_homo=True, # If True: build/use a VP table even for
homogeneous arrays (PB-aware imaging)

# Antenna/aperture efficiencies for tsys-atm noise model // Eff. for 7-and 12m at various freqs
# can be found in
# the ALMA technical handbook page 147 (cycle 12 handbook)
eta_A=0.63, # Efficiency for large dishes (e.g. 12m)
eta_B=0.66, # Efficiency for small dishes (e.g. 7m)
spillefficiency=0.96, # Spillover efficiency
trx_K=None, # Receiver temperature (K) ; None => auto from
ALMA band & center_freq
correfficiency=0.88, # Correlator efficiency
waterheight=2000.0, # Water vapour scale height (m)

pb_model="alma", # 'alma', 'airy', or 'gauss' ('alma' is highly
recommended, but you can try 'airy' as well)

# --- Validation / diagnostic "buttons" ---
validate_noise_theory=True, # Run noise-validation imaging?
noiseval_field="0", # FIELD id used for noise validation (keep as 0,
only 0 is currently supported)
noiseval_do_groups=True, # For heterogeneous arrays: also do A/B/cross
groups?
print_vptable=False, # If True: prints the combined vptable with the
PB mapping and which antennas use which PB.

image_baseline_groups=True, # Also image A/B/cross groups for the main run?

# --- Logging / cleanup ---
log_file="pipeline_test.log", # Log file (None = stdout only)
clean_project_base=True, # Remove previous outputs before run

# --- RMS measurement settings (used in measure_image_stats) ---
# PS: note that when FoV << PB, aliasing will occur, for accurate RMS, one should mask the
edges of the image!
rms_mode="auto", # 'full' = whole image, 'pb' = PB-limited region,
'auto' = apply masks below (PB mask + geometric masks)
rms_use_pb=True, # If True: apply PB mask (keep pixels where PB >=
rms_pb_min)
rms_pb_min=0.05, # Primary-beam cutoff for RMS regio
# --- Geometric masking controls ---
rms_center_frac=0.1, # Exclude a central box of this fractional size
# (e.g., 0.10 -> remove inner 10% area; useful to
mask bright source cores)

```

```

rms_keep_center_frac=0.8,          # Keep only a central box of this fractional size
(crops away noisy edges).          # If both rms_keep_center_frac AND
                                     # rms_center_frac are set:
                                     # -> RMS is computed in an ANNULUS-LIKE region:
                                     # central box defined by keep_center_frac
                                     # MINUS the inner excluded box defined by
                                     # center_frac.
# Summary of combinations:
#   Only rms_center_frac set:      mask *only* the inner core.
#   Only rms_keep_center_frac set: use *only* central part of image; discard edges.
#   Both set:                      keep middle region but punch out the inner core
#                                   (annulus / donut mask).
#   Neither set:                   no geometric masking (PB mask only, if enabled).
rms_nsigma_clip=5.0,               # Sigma clipping for robust RMS estimation
rms_max_points=2_000_000,          # Max number of pixels sampled for RMS
)

```

In practice:

- Paths, array configuration, and frequency setup control which ALMA config, declination, and bandwidth are used.
- Imaging/sky options set the FoV, pixel sampling, and which built-in source that is to be used (or your own supplied source, as a fits file).
- `tclean` parameters (weighting, robust, taper, niter) set the imaging style and deconvolution depth.
- Noise and PB settings define atmospheric conditions, aperture efficiencies, and the analytic PB model.
- Validation flags toggle noise-validation imaging and baseline-group checks.
- RMS settings control how image noise is measured (PB mask, central exclusion, sigma-clipping).

5 THERMAL NOISE IN A HETEROGENEOUS INTERFEROMETER

In an heterogeneous array, the antennas have different diameters and therefore different sensitivities. This is usually expressed via the system-equivalent flux density (SEFD) of each antenna, and the thermal noise on a baseline i – j scales roughly as⁸

$$\sigma_{ij} = \frac{1}{\eta_s} \frac{\sqrt{\text{SEFD}_i \text{SEFD}_j}}{\sqrt{2 \Delta \nu t_{\text{int}}}}, \quad (7)$$

where η_s is the system efficiency (typically $\eta_s \simeq 0.88$ in ALMA/CASA (ALMA Observatory, 2024)). For a homogeneous baseline D – D , this reduces to

$$\text{SEFD}(D) = \eta_s \sigma_{DD} \sqrt{2 \Delta \nu t_{\text{int}}}, \quad (8)$$

Larger dishes (lower SEFD) give smaller σ_{ij} ; baselines involving smaller dishes are noisier.

In our pipeline, we do not manually compute σ_{ij} for each baseline. Instead, we let CASA’s noise machinery generate the vis-domain noise per antenna (via atmospheric parameters and the ATM model), which implicitly produces the correct baseline-dependent σ_{ij} for heterogeneous arrays.

5.1 How we add noise: noise first, then skymodel

The heterogeneous simulation follows this order:

1. Create an empty measurement set with the desired array configuration, pointing pattern, and spectral setup (`makeMSFrame`).
2. Attach per-antenna primary beams in a VP table (`build_per_antenna_pb_vptable`), so that later imaging and prediction know about the different dish sizes.
3. Add thermal noise in the visibility domain before any sky signal is introduced:

⁸See <https://science.nrao.edu/facilities/vlba/docs/manuals/oss/bsln-sens>

* If `noise_mode='atm'`, we call `addNoiseSim(msname=..., pwv_mm=..., t_ground_K=..., ...)`, which uses an atmospheric / system model to populate `DATA` with complex Gaussian noise.

After this step, `DATA` contains noise-only visibilities with the correct heterogeneity. Importantly, because the visibilities are still noise-only at this stage, the noise can be rescaled to represent *any* desired aperture efficiency pattern. The helper routine `_apply_efficiency_scaling_tsysatm` adjusts each baseline (i, j) so that its RMS scales as

$$\sigma_{ij} \propto \frac{1}{\sqrt{\eta_i \eta_j}},$$

where η_i denotes the assigned aperture efficiency of antenna i . Since the scaling is applied prior to inserting the sky model, the signal remains intact and only the thermal noise is modified. This makes it possible to prescribe physically plausible efficiency variations (e.g., reduced efficiencies for larger dishes) and thereby simulate heterogeneous arrays more realistically.

4. Optionally, we estimate effective SEFDs per dish diameter from the noisy visibilities (`estimate_sefd_by_diameter`), as a sanity check that the injected noise scales as expected with dish size.

5. We then predict the sky model into the `MODEL_DATA` column:

- Import the input FITS image into CASA (`importfits`).
- Regrid / reshape it to match the MS (`make_startmodel_match`).
- Use `predictImager` with `gridder='mosaic'` and the per-antenna VP table to compute model visibilities in the `MODEL_DATA`.

6. Finally, we add the sky model to the noisy visibilities:

```
tb.open(ms_out, nomodify=False)

data = tb.getcol("DATA")           # noise-only
model = tb.getcol("MODEL_DATA")    # noiseless model visibilities

data += model                      # DATA <- noise + signal

tb.putcol("DATA", data)
tb.close()
```

Because the noise was generated (and, if desired, efficiency-scaled) *before* adding the sky model, the final `DATA` column represents a realistic observation: the true model visibilities plus thermal noise whose variance correctly reflects the heterogeneous per-antenna sensitivities.

5.2 Expected image-domain noise for baseline groups

For noise analysis we split the array into baseline groups:

- **A – A** : baselines between the larger dishes,
- **B – B** : baselines between the smaller dishes,
- **A – B** (or 'cross') : cross baselines.

For each group g , the pipeline computes the per-visibility noise directly from the Measurement Set using the routine `load_sigma_vis_and_N()`, which extracts the unflagged real and imaginary parts of the `DATA` column and evaluates

$$\sigma_{\text{vis},g} = \frac{1}{2} [\text{std}(\Re V) + \text{std}(\Im V)], \quad (9)$$

and denote by N_g the number of unflagged complex visibility samples.

Image noise for a single baseline group. Under natural weighting and in the absence of signal, a dirty image pixel is a weighted average of N_g independent visibility samples, each with variance $\sigma_{\text{vis},g}^2$. For equal weights within a group, the variance of the resulting image plane is therefore

$$\sigma_{\text{im},g} \simeq \frac{\sigma_{\text{vis},g}}{\sqrt{N_g}}, \quad (10)$$

which is the same expression used internally in both `compute_theoretical_rms()` and `hetero_checkVals()` when predicting group-wise image noise. This is a direct consequence of the radiometer equation and the fact that natural weighting performs an inverse-variance average of the visibilities.

Combined image noise for a heterogeneous array. An image made from all baselines jointly (A–A, B–B, and A–B) can be viewed as the optimal inverse-variance combination of the individual group images. Since the image obtained from group g has variance $\sigma_{\text{im},g}^2$, the combined variance is

$$\frac{1}{\sigma_{\text{im},\text{all}}^2} = \sum_{g \in \{A,B,\text{cross}\}} \frac{1}{\sigma_{\text{im},g}^2} = \sum_g \frac{N_g}{\sigma_{\text{vis},g}^2}, \quad (11)$$

where the second equality follows from Eq. (10). This expression is implemented directly in `compute_theoretical_rms` which accumulates the terms $N_g/\sigma_{\text{vis},g}^2$ for all baseline groups and evaluates $\sigma_{\text{im},\text{all}} = W_{\text{tot}}^{-1/2}$ with $W_{\text{tot}} = \sum_g N_g/\sigma_{\text{vis},g}^2$.

where the second equality follows from Eq. (10). Thus the expected rms noise of the full heterogeneous imaging run is

$$\sigma_{\text{im},\text{all}} = \left(\frac{N_A}{\sigma_{\text{vis},A}^2} + \frac{N_B}{\sigma_{\text{vis},B}^2} + \frac{N_{\text{cross}}}{\sigma_{\text{vis},\text{cross}}^2} \right)^{-\frac{1}{2}}. \quad (12)$$

Equation (10) describes how visibility-domain noise propagates into naturally weighted images, while Equation (11) is the inverse-variance combination used within `compute_theoretical_rms()` and subsequently validated by `run_noise_validation_single_field()` and `hetero_checkVals()`. These relations ensure full consistency between the analytic expectations and the numerical results obtained from the simulated Measurement Sets.

5.3 Noise validation: from visibilities to images

To check that both the noise injection and the theoretical formulas are correct, we run a noise dedicated noise-validation pipeline:

1. Create a noise-only copy of the MS:

- Copy the simulated MS to `base_noise_ms`.
- Subtract the model visibilities from the data:

$$\text{DATA} = \text{DATA} - \text{MODEL_DATA},$$

so DATA now contains only the injected thermal noise.

2. Form baseline-group sub-MSs (A, B, Cross) from this noise-only MS using `mstransform`.

3. Compute theoretical image RMS:

- For each group and for the full MS, measure $\sigma_{\text{vis},g}$, and N_g from DATA.
- Compute Eq.10, and Eq.11.

4. Measure image RMS from CASA:

- For each MS (ALL, A, B, Cross) run `tclean` with:.
- `weighting='natural'`,
- `gridder='mosaic'`,
- `niter=0`,
- the same VP table as in the simulation,

This produces pure-noise residual images. We can then use `get_rms_casa` to measure the image RMS. Note that when the FoV is much smaller than the PB, as when observing at high resolution, some PB-aliasing effects will occur and can manifest at the edge of the image/FoV. Therefore, if requiring an $\text{FoV} \gg \text{PB}$, try to image at a slightly larger FoV than what is desired, and then mask out 10 – 20% of the image edges. This seems to get rid of all/most of the aliasing effects.

5.4 Definition of Validation Metrics

All validation tables in this work use a common set of visibility- and image-domain diagnostics. For each baseline group g , we compute:

- **VisStd_{sim}**: the measured per-visibility RMS from the simulated DATA column, using $\sigma_{\text{vis}} = \frac{1}{2}[\text{std}(\Re V) + \text{std}(\Im V)]$.
- **VisStd_{norm}**: the visibility RMS normalized to the reference baseline type, highlighting the expected noise scaling between dish sizes⁹.
- **VisStd_{calc}**: the analytic prediction based on SEFD scaling, $\sigma_{ij} \propto (\eta_i \eta_j)^{-1/2}$, or CASA's atmospheric model when applicable.
- **#DataPts**: the number of unflagged visibilities in group g , which sets N_g in the radiometric relation $\sigma_{\text{im},g} \simeq \sigma_{\text{vis},g} / \sqrt{N_g}$.
- **Weights** (w_{calc} , w_{sim}): analytic and MS-reported natural weights, which should satisfy $w \propto 1/\sigma_{\text{vis}}^2$.
- σ_{calc} : the predicted image-plane noise, combining baseline groups through the inverse-variance relation $\sigma_{\text{im},\text{all}}^{-2} = \sum_g N_g / \sigma_{\text{vis},g}^2$.
- σ_{meas} : the empirically measured RMS from the pure-noise dirty image.
- **meas/calc**: the ratio $\sigma_{\text{meas}}/\sigma_{\text{calc}}$, quantifying the agreement between simulation and analytic expectation. Values near unity indicate excellent consistency.

⁹Note that agreement between $\text{VisStd}_{\text{sim}}$, $\text{VisStd}_{\text{calc}}$, and the corresponding natural weights is expected *within each baseline type*, but not necessarily *across* baseline types. When all antennas have the same aperture efficiency (e.g. a homogeneous 12 m array), all baselines share the same thermal noise level and thus $\text{VisStd}_{\text{sim}} \approx \text{VisStd}_{\text{calc}}$ and $w_{\text{sim}} \approx w_{\text{calc}}$ for every group. For heterogeneous arrays (e.g. 12 m+7 m or 25 m+12 m), different antenna pairs have intrinsically different SEFDs, so visibility noise scales as $\sigma_{ij} \propto (\eta_i \eta_j)^{-1/2}$. In this case the baseline groups are expected to exhibit distinct values of VisStd and of the natural weights, consistent with the analytic SEFD-based predictions.